

DLAM Final Report 2026: Multivariate Time Series Forecasting of Hourly Operational Load

September 2026

Chaithanya Kalyan Bhuma, 3290040
chaithanya.bhuma@stud.tu-darmstadt.de
Marcel Schlappner, 2556785
marcel.schlappner@stud.tu-darmstadt.de

Gerhard Peringer, 2439608
gerhard.peringer@stud.tu-darmstadt.de
Paul Ferdinand Safari, 3863888.
paul.safari@stud.tu-darmstadt.de

1 Introduction

Accurate demand forecasting is important for operational planning and resource allocation. In this project, we study multivariate time series forecasting with the goal of predicting hourly operational load.

The benchmark contains 4320 hourly training observations (1 Jan–29 Jun 2023) for 96 units and 22 known future covariates, followed by a hidden validation period scored on a public leaderboard and a private test period used for final evaluation.

We compare several forecasting approaches from different model families, including Transformer-based and dense architectures [1, 2]. Based on these experiments, we further develop the most promising approaches and investigate how architectural choices, covariates, and preprocessing affect forecasting performance. We additionally evaluate the models on a second dataset to examine their behavior outside the benchmark domain.

2 Related Work

2.1 iTransformer

The iTransformer[1] is a Transformer-based model developed specifically for time series forecasting. Its main difference compared to traditional Transformer architectures lies in how the input data is represented. Instead of treating individual time steps as tokens, iTransformer represents each variable of a multivariate time series as a separate token containing its complete historical sequence. The self-attention[3] mechanism can then learn relationships between the different variables, while the feed-forward network captures temporal patterns within each variable. This approach makes iTransformer particularly suitable for multivariate forecasting tasks, where dependencies between different variables can provide useful information for predicting future values.

2.2 PatchTST

PatchTST[2] is a Transformer-based model designed for time series forecasting. Instead of processing each individual time step separately, the model divides the historical time series into smaller segments, called patches. Each patch is represented as a token and processed by the Transformer. This reduces the number of tokens and allows the model to focus on patterns that occur over multiple time steps rather than on individual observations. By applying self-attention[3] across these patches, PatchTST can capture both short- and long-term dependencies in the data. This makes the model well suited for forecasting tasks where recurring patterns and longer temporal relationships are important.

2.3 TiDE

TiDE (Time-series Dense Encoder)[4] is a forecasting model that uses a multilayer perceptron (MLP) based encoder-decoder architecture instead of self-attention. The encoder processes the historical time series together with available covariates and transforms them into a compact representation. The decoder then uses this representation to predict future values over the desired forecasting horizon. By relying mainly on dense layers, TiDE provides a comparatively simple and computationally efficient alternative to Transformer-based approaches. At the same time, it is designed to capture nonlinear relationships and long-term dependencies in time series data, making it suitable for a wide range of forecasting tasks.

2.4 N-BEATSx

N-BEATSx[5] is an extension of the N-BEATS architecture[6] designed for time series forecasting. The model is based on fully connected neural networks and does not rely on recurrent or attention-based components. It processes historical observations through several stacked blocks, where each block learns to explain part of the input and contributes to the final forecast. Compared to the original N-BEATS model, N-BEATSx can additionally include exogenous variables, such as external or known future information, in the forecasting process. This allows the model to consider factors beyond the historical target values and makes it suitable for forecasting tasks where additional information may influence future developments.

3 Method

3.1 Setup and the lag constraint

The target $y_{u,t}$ is observed for 96 units over hours $t \leq 4319$. Validation asks for $t \in [4320, 4655]$ and the private test for $t \in [4656, 4991]$. Along with the target we get 22 covariates whose values are known for the whole future.

The gap between train and test is what shapes the design. At test time the last observed target is hour 4319 and the first hour we have to predict is 4656. A lag of k hours is usable only if $t - k \leq 4319$ for every test hour t , which means $k \geq 672$. Anything shorter lands inside the hidden validation fortnight. It would score fine on the leaderboard and then fail on the private split, so we reject shorter lags in code.

Legal lags are not worth much anyway. A lag-672 copy scores 43.34 WAPE against 32.06 for a plain hour-of-week profile, and the autocorrelation at 672 h is 0.38. Lag features also cost 41% of the training windows, because early windows have nothing to look back at. So we dropped target history entirely. The model reads covariates and calendar only. One consequence is that it is the same model at validation and at test, while a lookback model sees a two-week-old past at test time.

3.2 Profile and residual

Everything the network predicts is a correction to a fixed table.

Before training we compute, for each unit, the median target at each of the 168 hours of the week, where $\text{how}(t) = 24 \cdot \text{dayofweek}(t) + \text{hour}(t)$. This gives a 96×168 table $P_u[\text{how}]$. We use the median and not the mean because WAPE is an L1 metric and these series have spikes that would pull a mean upward. We also store IQR_u , the interquartile range of $y - P_u[\text{how}]$ over the training rows, as a per-unit residual scale.

Both are ordinary statistics, not learned parameters. They are fitted on training rows only and saved in the checkpoint, so inference rebuilds exactly what training used. The profile on its own scores 32.06 WAPE against 54.50 for naive last value.

The network outputs a residual r_t per horizon position and one scalar ℓ per window. The forecast is

$$\hat{y}_{u,t} = \max(0, e^\ell P_u[\text{how}(t)] + \text{IQR}_u r_t). \quad (1)$$

Dividing by IQR_u puts a unit with typical load 3 and a unit with typical load 400 on the same scale, so one set of weights serves all 96 units. The profile also goes in as an input channel, but standardised inside the window, so the network sees the shape of the week and not the magnitude. The magnitude is added back afterwards by Eq. (1).

3.3 Inputs

One token per horizon hour, 336 tokens per window, one unit at a time. Each token has 55 channels:

- 21 covariates, z-scored (the 22 provided minus `trend`, see Section 3.6);
- 21 missing masks. A NaN means the covariate is unavailable, so we set it to zero and raise the flag, which lets the network tell a real zero from a missing value;
- the profile row, standardised within the window;
- 8 cross-unit means: for each hour, the mean over all 96 units of queue pressure, network pressure, event load, shock risk, the three risk forecasts and maintenance;
- 4 day-of-month harmonics: $\sin$ and $\cos$ of $2\pi d/D$ and $4\pi d/D$ for day d of a D -day month.

Two of those blocks need a reason. The cross-unit means are there because the level swing is system-wide. One unit's covariate is noisy, but the average of the same quantity over 96 units is a clean version of the same signal, and a window that only covers one unit cannot see it.

The day-of-month harmonics are there because the profile is indexed by hour of week while the level cycle has a period of one calendar month, and the two calendars do not line up. The 25th is a Tuesday in April, a Thursday in May and a Sunday in June. No weekly table has a cell that means "the 25th". Two harmonics cover one bump per month and two bumps per month, which is enough for both datasets we tried.

3.4 Architecture

A linear layer maps the 55 channels to $d = 192$ (Fig. 1). A 32-dimensional unit embedding e_u is used in two ways. It is projected and added to the hidden state, and it also produces a FiLM scale and shift, $h_t \leftarrow h_t \odot (1 + \gamma_u) + \beta_u$. The additive part lets a unit sit higher or lower overall. The multiplicative part lets a unit react more or less strongly to a covariate, which the additive part cannot do [7]. Both are initialised to zero, so the model starts at the identity.

Hour-of-week and sinusoidal position embeddings are added. Then three pre-LN encoder layers [3], 8 heads, feed-forward 768, dropout 0.3, attending across the 336 positions of one unit. Attention helps here because the quantity that drives the level is slow. It is roughly a two-week average of the pressure covariates, and averaging over positions is what attention does.

Two heads read the encoder output. A per-position head gives r_t . A second head reads the mean-pooled sequence and gives one scalar ℓ , which multiplies the profile. A level shift over a whole fortnight then costs the network one number instead of 336 residuals that all have to agree. The model has 1.40 M parameters.

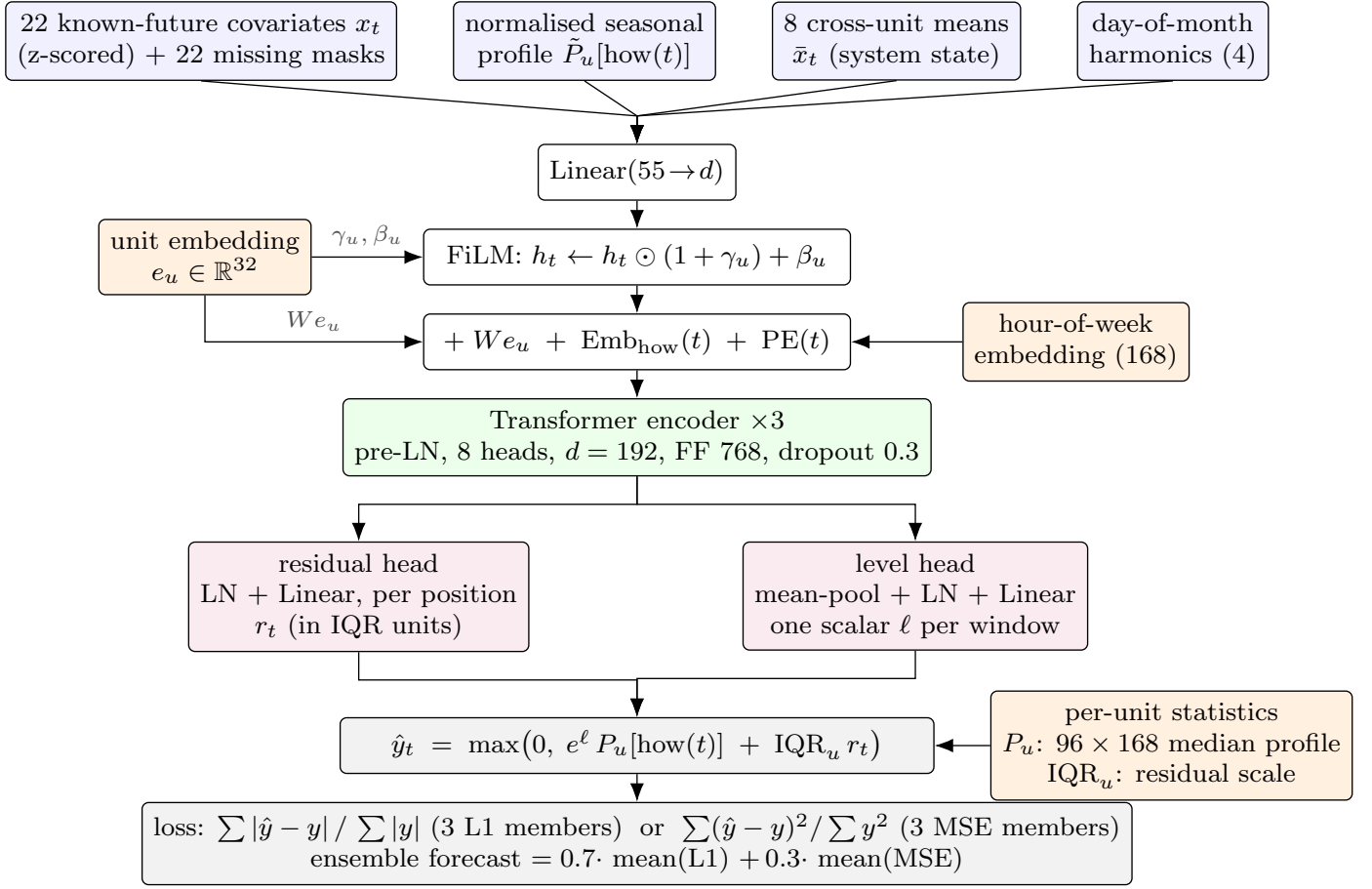


Figure 1: The residual encoder, applied per unit u to the 336 horizon positions. Only known-future information enters. The unit embedding conditions the covariate path through FiLM; two heads produce per-position residuals and one multiplicative level per window; the forecast is composed on the per-unit profile and residual scale. L1- and MSE-trained members are blended 0.7 / 0.3.

3.5 Objective, training and ensemble

L1 members minimise batch WAPE, $\sum |\hat{y} - y| / \sum |y|$, over observed hours. This targets the conditional median, which is the WAPE-optimal forecast, and it weights units by volume the same way the metric does.

The leaderboard also ranks on MSE and RMSE, so we train MSE members too. They minimise $\sum (\hat{y} - y)^2 / \sum y^2$, which targets the conditional mean. Mixing the two helps for a simple reason. At the median, the derivative of the expected absolute error is zero and the loss has a flat quadratic bottom, so a small move away costs almost nothing. The derivative of the expected squared error at the median is $2(\text{median} - \text{mean})$, which is not zero when the data is skewed, so the same small move gains linearly on MSE. We picked the weights on the local holdout with a composite proxy over the six metrics and settled on 0.7 for the L1 group and 0.3 for the MSE group.

Training uses AdamW [8] with lr 3×10^{-4} and weight decay 10^{-2} , a one-cycle schedule [9], batch 64, bf16, gradient clipping 1.0, and sliding 336-hour windows at stride 6, which gives about 58k windows per epoch. Every hour appears in 56 windows, so the optimum is reached after 6 to 10 epochs and the model degrades after that.

The shipped ensemble has six members: three L1 (one holdout-selected at epoch 5 of a 10-epoch schedule, two trained on all 4,320 h for 10 epochs with seeds 1 and 2) and three MSE (two holdout-selected after 3 epochs, one trained on all data for 4 epochs).

3.6 What the diagnostics found

Our first residual encoder (v2: same backbone, 45 input channels, **trend** included) scored 15.21 on the leaderboard from 14.06 locally. Instead of tuning it we measured where the error was. Each change below answers one measurement.

The level moves in a monthly cycle.

The median of y/P_u across units, taken per week, swings between 0.73 and 1.23 with a period of one calendar month (Fig. 2, left). The autocorrelation is -0.79 at two weeks and $+0.63$ at four, and 54% of the variance of the 96 unit levels sits on one principal component, so the swing is shared across units.

The covariates already contain it.

Weekly means of the queue pressure and network pressure forecasts correlate 0.98 with the global level. A ridge regression on the 13 weekly covariate means predicts it with leave-one-out $R^2 = 0.96$, and a unit's own weekly means predict its level with week-blocked $R^2 = 0.89$. So the signal was in the inputs at every hour and the network was under-reacting. That is what the cross-unit means and the level head are for. Copying a recent level does not work: the fortnight before the holdout sat at 0.91 and the holdout at 1.10.

	WAPE	MAE	MSE	RMSE	MAPE	sMAPE	bias%
<i>Local holdout, 16–29 Jun 2023 (statistics fitted before it; $d = 192$, 3 layers, stride 6, seed 0, 6 epochs unless stated)</i>							
Naive last value	54.50	–	–	–	–	–	–
Lag-672 copy (largest legal lag)	43.34	–	–	–	–	–	–
Seasonal profile (unit×hour-of-week median)	32.06	3.424	21.98	4.688	40.54	35.64	−13.0
v2 residual encoder (with trend)	13.93	1.487	6.86	2.618	16.53	15.92	−4.7
v2 without trend	15.18	1.621	8.19	2.862	18.36	17.32	−7.2
A: v2 + calendar, cross-unit means, level head	13.63	1.456	7.58	2.753	17.00	15.28	−4.2
B: A without level head	13.72	1.465	7.77	2.787	17.12	15.33	−4.6
C: A without trend (final L1 member)	13.43	1.434	7.12	2.668	16.13	15.08	−5.5
C, 10-epoch schedule (best epoch 5)	12.98	1.387	6.60	2.569	14.80	14.57	−5.8
D: C with MSE objective (final MSE member)	13.89	1.484	6.18	2.485	17.38	15.56	−1.3
Blend 0.7 L1 / 0.3 MSE (two members each)	12.93	1.381	6.21	2.491	15.35	14.45	−4.5
<i>Public validation leaderboard, 30 Jun–13 Jul 2023</i>							
Naive last-value baseline (provided)	48.10	–	–	–	–	–	–
Seasonal mean baseline (provided)	34.41	–	–	–	–	–	–
Ours, v1: first covariate-driven Transformer	16.33	–	–	–	–	–	–
Ours, v2: residual encoder	15.21	1.675	8.68	2.947	17.21	17.01	–
Ours, final: six-member ensemble	12.89	1.419	6.96	2.638	15.08	14.32	–

Table 1: Benchmark results (WAPE, MAPE, sMAPE in percent; bias = $\sum(\hat{y} - y) / \sum|y|$). Early lookback-based attempts (PatchTST- and iTransformer-style models under the no-history protocol) scored 46–47 WAPE on the leaderboard and are omitted.

trend was a crutch.

trend is a global linear ramp with training range $[-1.73, 1.03]$, holdout $[1.03, 1.27]$, validation $[1.27, 1.50]$ and test further out. The network was using it to memorise which week it was in, and through that, which phase of the month. Removing it from v2 cost 1.25 WAPE (13.93 to 15.18) and pushed the bias to -7.2% . After we added the day-of-month harmonics and the cross-unit means, which give the same information in a range that is always valid, removing **trend** gained 0.20 instead. The local-to-leaderboard offset also went from $+1.15$ to -0.04 , which is the part we trust most.

Spikes hold most of the squared error.

Hours more than two residual IQRs above the profile are 2.6% of hours and 5.4% of $\sum|y|$, but they hold 15.5% of the absolute error and 36.9% of the squared error, and they are predicted 40% low. They are not random. The top decile of `event_load_forecast`, `demand_forecast` and `shock_risk` has a spike rate of 9 to 10% against a 2% base rate, with Cohen’s d of 0.70, 0.64 and 0.62. Under L1 the conditional median correctly ignores a 10% event. Under MSE it does not, which is where the MSE members come from.

The model under-predicts.

v2 was low by 5.1% of $\sum|y|$ (-2.2 points from spike hours, -2.2 from ordinary hours, -0.9 from maintenance), and 86 of 96 units were biased low. This is what L1 does on right-skewed data, and it is the second reason for the two-objective ensemble.

3.7 Protocol

The local holdout is the last 336 h of training (16 to 29 Jun). Statistics are fitted on the data before it, never on the holdout itself, and epochs are selected on it. A three-fortnight variant (19 May to 29 Jun, pooled) gave a period-to-period spread of about 0.02 WAPE, so single-window differences below about 0.005 are noise. We report all six leaderboard metrics locally because the ranking is composite.

3.8 Results

Table 1 gives the local holdout and the public leaderboard. Read paired on the holdout, the level head is worth 0.09 WAPE and also lowers MSE. The calendar and cross-unit features are worth 0.30 with **trend** present, and they make **trend** removable, which is worth another 0.20. A 10-epoch schedule reaches 12.98 at epoch 5 but ends at 13.27, so the full-data members use that schedule and the epoch-5 network is kept as its own member. The MSE member trades 0.46 WAPE for 13% lower MSE and a bias of -1.3% instead of -5.5% . The 0.7 / 0.3 blend beats both of its members on WAPE as well as on the composite, partly from the median-mean trade above and partly from ordinary variance reduction. Seeds, full-data members and the ensemble carried that to 12.89 on the leaderboard, which is 0.04 below the local figure, where v2 had been 1.15 above it.

Two things explain the error that is left. The L1 models became more confident that spikes will not happen once they could see the system state (spike-hour predictions of 12.4 to 12.6 against 13.2 for v2, with a true spike mean of 21.9). That is right for the median and wrong for MSE, which is where the MSE member’s 14.8 helps. And the ordinary-hour WAPE of the best single model is 12.0 with a residual IQR of half the median, so most of the rest is noise the covariates do not resolve.

3.9 What did not matter

Capacity: 0.64 M, 1.4 M and 3.2 M parameters all landed within 13.95 to 14.86 WAPE, with the largest being the worst. Window density: going from stride 24 to stride 6 changed WAPE by 0.06. Weight averaging: EMA lost at every epoch under the one-cycle schedule. Legal lag features: 672 and 840 h both lost to the profile and cost 41% of the training windows.

4 Experiments on the benchmark dataset

4.1 Explored Models

4.1.1 iTransformer

The main challenge when applying iTransformer [1] to our setting was the univariate nature of the forecasting task. Our original implementation transformed the complete 168-hour history into a single token. As a result, the self-attention mechanism had no different variables or representations between which it could model dependencies. To address this limitation, we represented the input through three complementary views: the original time series, the differences between consecutive observations, and the differences to observations 24 hours earlier. Each view was normalized independently and embedded as a separate token. Learnable type embeddings were further used to indicate which temporal representation each token corresponded to. The resulting tokens were processed by the Transformer and subsequently combined using attention-based pooling.

We also changed the prediction strategy itself. Rather than generating the next 24 absolute values directly, the adapted model estimates corrections to the most recently observed 24-hour profile. The intention was to provide the model with the recurring daily pattern as a starting point while using its capacity mainly to learn deviations from this pattern. However, this approach did not result in better forecasts. The WAPE increased from 47.39 for the original implementation to 51.99 after the modifications. Thus, the additional temporal representations and seasonal correction mechanism increased the complexity of the approach without providing a corresponding improvement in forecasting accuracy on our dataset.

4.1.2 PatchTST

For PatchTST [2], we focused primarily on changing the forecasting target from absolute values to daily residuals. For each observation, the difference to the corresponding value 24 hours earlier was calculated and used as input to the model. PatchTST was then trained to estimate future deviations from the previous day rather than reconstructing the complete future sequence. The final prediction was obtained by adding these estimated deviations to the latest observed daily profile. This formulation was motivated by the strong daily structure of the data and was expected to provide an easier prediction target.

Further changes were made to the internal configuration of PatchTST. Instead of averaging the patch representations before generating the forecast, all patches were retained, allowing the prediction head to access their individual temporal information. Layer Normalization with pre-normalization was used in place of the previous Batch Normalization configuration. We additionally increased the model dimension from 64 to 128 and set the feed-forward network size to 256 to provide additional model capacity.

In practice, these changes had a strongly negative effect on forecasting accuracy. The original PatchTST achieved a WAPE of 45.90, whereas the modified configuration produced a WAPE of 112.72. This result was not only substantially worse than the original PatchTST but also failed to outperform the baseline model. Consequently, the residual-based formulation and architectural adjustments were not retained for the final model selection.

4.1.3 N-BEATSx

Based on N-BEATSx [5], we implemented an N-BEATSx-inspired forecasting model adapted to the structure of the benchmark dataset. Instead of processing all inputs jointly, we used separate encoders for the historical target values, past covariates, and known future covariates. The past and future covariate branches were equipped with temporal convolutional networks (TCNs) [10] to capture temporal patterns at different scales. In addition, learned series embeddings and static features were included to represent differences between the individual units.

The model uses a 168-hour target history and produces forecasts in 24-hour blocks. Within each forecasting block, the encoded target and covariate representations are combined and processed by stacked residual blocks following the residual forecasting principle of N-BEATS [6]. Each block produces both a backcast, which is subtracted from the remaining target representation, and a forecast contribution that is added to the overall prediction. Longer forecasting horizons are obtained by repeatedly applying the model and feeding its predictions back into the target history.

Model Development

The architecture was developed incrementally. Starting from a simpler target-based model, we successively introduced exogenous covariates, learned series embeddings, covariate standardization, missing-value indicators, and static features. Separating past and future covariates and processing them with dedicated temporal convolutional encoders [10] produced one of the largest improvements during development.

We also evaluated several architectural extensions, including target-side temporal convolutions, feature gating, FiLM conditioning [7], multi-scale target representations, and rollout-aware training. These modifications did not provide consistent improvements across validation windows and random seeds and were therefore not retained.

We further tested different forecasting horizons, target normalization using RevIN [11], capacity normalization, seasonal residual formulations, exponential moving averages, and different optimizer and learning-rate schedules. AdamW [8] was evaluated in addition to Adam but did not provide a consistent advantage.

For the training objective, we primarily used Huber loss [12]. We additionally evaluated L1 loss, composite metric-based objectives, and combinations of Huber loss with sMAPE [13]. While some alternatives improved individual runs or individual metrics, they did not consistently outperform the selected objective across validation windows and random seeds.

Hyperparameters were selected using rolling-window validation on the released training data. The strongest configurations were subsequently evaluated using multiple random seeds to reduce the influence of initialization variance.

Rossmann, 100 stores, 28-day horizon	WAPE Jun	WAPE Jul	WAPE	RMSPE	MAE	bias%
Naive last value	32.01	31.77	31.89	55.97	2112	+23.3
Store×day-of-week profile	18.18	17.05	17.63	24.99	1167	−0.3
Full model (L1)	10.42	11.63	11.01	16.51	729	−3.7
without level head	10.61	11.69	11.14	15.92	738	−5.3
without FiLM	10.29	11.93	11.10	16.60	735	−1.5
without day-of-month harmonics	10.60	11.72	11.15	16.16	738	−4.0
without cross-store means	10.39	11.22	10.80	15.82	715	−4.0
MSE objective	10.70	11.23	10.96	15.93	726	−4.2

Table 2: Rossmann transfer, open days only. A replication at stride 1 and 8 epochs gave 10.80 for the full model, 10.29 without cross-store means and 10.31 with the MSE objective; other ablations within 0.2.

4.1.4 TiDE

Our first version predicted 24 hours at a time and repeated this 14 times. The forecasts became flat towards the end, because each call feeds the previous predictions back as input. Since our dataset provides all covariates for the prediction period, we did not need this loop and switched to predicting all 336 hours at once.

We then compared three sources for the normalization statistics: per-series (13.44), none (13.54), and per-window as in RevIN [11] (13.95). Per-series was best, but only by 0.7 to 3.8 percent with a single seed, which is within random variation. We kept it because it does not depend on which window precedes the forecast. We also removed the trend column, since all its values in the prediction period lie above the training maximum and its correlation with the target is only 0.052.

Comparing lookback lengths gave 336 hours as the clear winner on validation (12.80 against 13.44), but the order reversed on the held-out split (15.48 against 15.89). We kept all lookbacks in the ensemble rather than picking a winner. Of three post-hoc corrections, an additive bias shift made things worse (12.43 to 12.70), since the L1 loss targets the median while the correction is based on the mean. This is the loss-metric mismatch listed as risk 4 in our proposal. Only a global factor of 1.015 was kept.

We split the data three ways and looked at the test portion once at the end. The validation score turned out to be 18 percent too optimistic. The naive last-value baseline reaches 51.76, the seasonal mean profile 32.05 on validation and 31.38 held out, and our calibrated ensemble 12.36 and 14.56. Series-specific embeddings from the proposal were not reached. All models used a single seed, so differences below roughly 3 percent are not meaningful.

5 Additional Dataset

Data, use case and setup

Rossmann Store Sales [14] records daily sales of 1,115 German drugstores from 1 Jan 2013 to 31 Jul 2015 (942 days), with store attributes. It is the structural twin of the benchmark: many related series, a strong day-of-week profile, level shifts driven by covariates that are known ahead (promotions, state and school holidays, opening days), static per-series attributes (store type, assortment, competition distance), and a well-known payday effect. Forecasting the next 28 days per store serves staffing and replenishment.

We sample 100 stores with complete history (934 qualify; seed 0). Closed days are set to missing and excluded from the loss and the metrics, which is the competition’s convention. The 20 features are all known ahead: day-of-week and month harmonics, Promo and Promo2 flags, school and state holidays, Open, log competition distance and a competition-open flag, and store-type and assortment one-hots. `Customers` is excluded because it is only observed after the fact.

The model is instantiated by changing the schema block of the core module only: daily grid, 28-step horizon, store×day-of-week profile, feature list, cross-store means of the promo and holiday flags, and closed-day handling. The training script is byte-for-byte the benchmark one. We use two 28-day folds (6 Jun to 3 Jul and 4 to 31 Jul 2015), statistics fitted before the first fold, WAPE over open days as the primary metric and RMSPE (the competition metric) as secondary, with $d = 128$, 3 layers, dropout 0.3, stride 7, 6 epochs, and the epoch selected on the pooled folds.

Results and transfer

The full model improves on the seasonal profile by 38% and on the naive baseline by 65% without any Rossmann-specific engineering (Table 2). The ablations tell a property-based story.

Removing the day-of-month harmonics costs 0.14 WAPE. Rossmann’s day-of-month effect has two peaks per month rather than the benchmark’s one (Fig. 2), so the second harmonic, which is unused on the benchmark, carries it here. The feature transfers even though the learned shape does not. Removing FiLM costs 0.09 and removing the level head costs 0.13, both within the roughly 0.2 noise of early-epoch checkpoints but consistent in sign with the benchmark.

Removing the cross-store means *improves* WAPE by 0.22 (0.51 in the replication). On the benchmark those means denoise covariates that are only 40 to 60% shared across units, whereas Rossmann’s promo and holiday flags are chain-wide, so the mean simply duplicates the flag and adds input dimensions for nothing. The MSE variant matches or beats the L1 model on WAPE while all L1 variants under-predict by 3 to 5%, as on the benchmark, which supports the two-objective ensemble as a design choice rather than a benchmark-specific hedge.

What transferred is the recipe: direct multi-step prediction from known-future covariates on a per-series profile base, per-series conditioning, and an L1 objective. What had to change was the schema. Each auxiliary component’s value tracks a measurable property of the data: month-cycle amplitude for the harmonics, unit-specific sensitivity for FiLM, synchronised level swings for the level head, and the cross-unit share of a covariate for the global means, which reverses

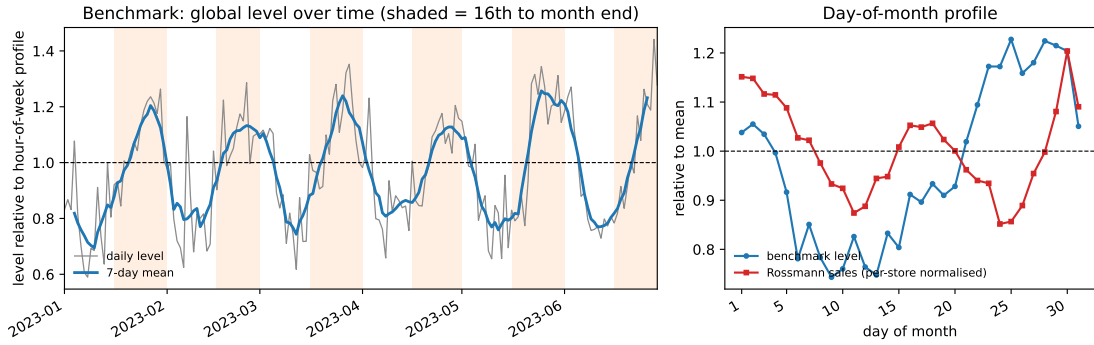


Figure 2: Calendar-month structure in both datasets. Left: benchmark level relative to the hour-of-week profile, median across the 96 units, 7-day mean in bold; second half of each month shaded. Right: day-of-month profile of the benchmark level and of Rossmann open-day sales, each normalised to its mean. The shapes differ, one cycle per month in the benchmark and two in Rossmann (paydays on the 1st and mid-month), but both are deterministic $\pm 15\text{--}20\%$ effects that a weekly profile cannot represent; the two-harmonic basis captures the benchmark’s shape with its first harmonic and Rossmann’s with its second.

sign when the covariates are already global.

Caveats: the optimum is reached within one to four epochs and degrades afterwards, and epochs are selected on the reported folds, so the absolute numbers are mildly optimistic and differences below 0.2 WAPE are not significant. The results are for a 100-store subsample and two summer folds and are not comparable to competition scores, which use all stores, a six-week horizon and tuned tree ensembles.

6 Conclusion

We compared models from several families on the same benchmark. The lookahead-based ones all have to read recent target history, which the private test split does not give them, and under the no-history protocol they scored between roughly 45 and 52 WAPE. The approach that worked drops target history completely and treats the task as a regression from known-future covariates onto a per-unit hour-of-week profile.

The final model reached 12.89 WAPE on the validation leaderboard, from 15.21 for our previous submission and 34.41 for the provided seasonal mean baseline. Most of that came from measuring the model’s own errors rather than from tuning: a calendar-month level cycle carried by the pressure covariates, a spike regime that holds 37% of the squared error, and a linear time index that let the network memorise the month phase instead of learning it. The fixes, namely calendar harmonics, cross-unit means, a multiplicative level head, dropping the time index and an ensemble over two objectives, are each small, and each was justified by a number before it was implemented. Because the model uses no lookahead, it is the same model at validation and on the private test, where methods that read the hidden fortnight will degrade.

Limitations: the residual noise bounds what the covariates can resolve; the L1 members under-predict right-skewed regimes and the ensemble only partly corrects this; training overfits quickly on per-series histories; and cross-series attention, quantile outputs for the spike regime and an explicit level model on weekly covariate means remain untested. On a covariate-driven benchmark, an hour of diagnostics on where the error lives was worth more than every capacity and regularisation sweep we ran.

References

- [1] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, “itransformer: Inverted transformers are effective for time series forecasting,” in *International Conference on Learning Representations* (B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, eds.), vol. 2024, pp. 11116–11140, 2024.
- [2] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *The Eleventh International Conference on Learning Representations*, 2023.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2017.
- [4] A. Das, W. Kong, A. Leach, S. K. Mathur, R. Sen, and R. Yu, “Long-term forecasting with tiDE: Time-series dense encoder,” *Transactions on Machine Learning Research*, 2023.
- [5] K. G. Olivares, C. Challu, G. Marcjasz, R. Weron, and A. Dubrawski, “Neural basis expansion analysis with exogenous variables: Forecasting electricity prices with nbeatsx,” *International Journal of Forecasting*, vol. 39, p. 884–900, Apr. 2023.
- [6] B. N. Oreshkin, D. Carpov, N. Chapados, and Y. Bengio, “N-beats: Neural basis expansion analysis for interpretable time series forecasting,” 2020.
- [7] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, “Film: Visual reasoning with a general conditioning layer,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, 2018.
- [8] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.

- [9] L. N. Smith and N. Topin, “Super-convergence: Very fast training of neural networks using large learning rates,” in *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications (SPIE)*, vol. 11006, 2019.
- [10] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018.
- [11] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, “Reversible instance normalization for accurate time-series forecasting against distribution shift,” in *International Conference on Learning Representations*, 2022.
- [12] P. J. Huber, “Robust estimation of a location parameter,” *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.
- [13] B. E. Flores, “A pragmatic view of accuracy measurement in forecasting,” *Omega*, vol. 14, no. 2, pp. 93–98, 1986.
- [14] Rossmann and Kaggle, “Rossmann store sales.” Kaggle competition, <https://www.kaggle.com/c/rossmann-store-sales>, 2015.